

Pointers & Dynamic Memory Allocation

Unit-02

Address Operator

Unit-02.1

In C++, the "address-of" operator, denoted by the ampersand symbol (&), is used to get the memory address of a variable. This operator returns the memory address where a variable is stored in the computer's memory.

Here's a basic example of how the address-of operator is used:

cpp

 Copy code

```
#include <iostream>
using namespace std;

int main() {
    int x = 5;
    cout << "The value of x: " << x << endl;
    cout << "The address of x: " << &x << endl;
    return 0;
}
```

Output:

less

 Copy code

The value of x: 5

The address of x: 0x7fff96e8e9dc // The specific address might differ

In the above example, `&x` retrieves the memory address of the variable `x`, which is then printed out using `cout`.

Things you should know...

1. **Essential for Pointers:** The address-of operator is fundamental when working with pointers in C++. Pointers store memory addresses, and the address-of operator retrieves these addresses, allowing manipulation of memory directly.
2. **Unchanged Value:** Applying the address-of operator to a variable does not modify its value. It only returns the memory address where the variable is stored.
3. **Size Consistency:** Regardless of the type of variable (e.g., int, float, custom class), the address-of operator always returns the same size of memory address. This size is typically determined by the architecture of the system (e.g., 32-bit or 64-bit).
4. **Pointer Initialization:** When declaring pointers, the address-of operator is often used to initialize them with the memory address of another variable. For example:

cpp

 Copy code

```
int x = 10;
int *ptr = &x; // ptr now points to the memory address of x
```

5. **Avoiding Pointer Arithmetic:** The address-of operator is useful for referencing variables directly, potentially avoiding complex pointer arithmetic that can be error-prone.
6. **Address Manipulation:** While typically used for accessing memory addresses of variables, the address-of operator can also be used for other purposes, such as passing the address of functions or class member functions.
7. **Cannot Be Applied to Constants or Literals:** The address-of operator cannot be applied to constants or literals because they do not have memory addresses. For example, you cannot do `&5` or `&"hello"`.
8. **Safety and Security Concerns:** While powerful, direct memory manipulation using pointers and the address-of operator can introduce bugs and security vulnerabilities if not used carefully, such as accessing invalid memory addresses or causing buffer overflows.
9. **Dereferencing:** The address-of operator is closely related to the dereferencing operator (`*`). While `&` retrieves the address of a variable, `*` dereferences a pointer, retrieving the value stored at the memory address it points to.
10. **Debugging Tool:** In debugging scenarios, printing out memory addresses using the address-of operator can be helpful for understanding memory layout, tracking down memory-related bugs, and optimizing code performance.

Pointer Operator

Unit-02.2

```
#include <iostream>
using namespace std;
int main(){
    int b = 2;
    int *a = &b;
    cout << a << endl;
    cout << *a << endl;
    return 0;
}
```

Let's Compare These Two
Blocks of Codes

```
#include <iostream>
using namespace std;
int main(){
    int b = 2;
    int* a = &b;
    cout << a << endl;
    cout << *a << endl;
    return 0;
}
```

What is different here?

```
#include <iostream>
using namespace std;
int main(){
    int b = 2;
    int *a = &b;
    cout << a << endl;
    cout << *a << endl;
    return 0;
}
```

```
int *a = &b;
```

Well, this is how you define.
Both ways are right.

```
int* a = &b;
```

```
#include <iostream>
using namespace std;
int main(){
    int b = 2;
    int* a = &b;
    cout << a << endl;
    cout << *a << endl;
    return 0;
}
```

What is happening here?

- `int*` is a type here, we call it an integer-pointer
- We see that the value of '`a`' holds the '**address of `b`**' which is '`&b`'
- Recommendation: Use `int* a = &b;` instead of using `int *a = &b;` Here semantics can be helpful, e.g., the first one you can read '`a`' as `int*` type, while the later one most of the beginners get confused and read it as '`*a`' as `int`

Do you know why this will fail?

```
#include <iostream>
using namespace std;
int main(){
    int b = 2;
    int* c;
    int* a = &b;
    c = a;
    c = 23;
    cout << "b = " << b << endl;
    return 0;
}
```

* as address resolution Operator

Unit-02.3

What do you think will be
the output of line 9?

```
1  #include <iostream>
2  using namespace std;
3  int main(){
4      int b = 2;
5      int* c;
6      int* a = &b;
7      c = a;
8      *c = 23;
9      cout << "b = " << b << endl;
10     return 0;
11 }
```

Something is happening in
line-8

```
1  #include <iostream>
2  using namespace std;
3  int main(){
4      int b = 2;
5      int* c;
6      int* a = &b;
7      c = a;
8      *c = 23;
9      cout << "b = " << b << endl;
10     return 0;
11 }
```

So why? `*c = 23`, why not `c = 23`; We now know the answer. `*c` resolves the address to value, the value resolved is `int`, so you can basically use `int` i.e., 23 to replace whatever pointer 'c' points to

What do you think will be
the last value of 'a' will be?

```
1  #include <iostream>
2  using namespace std;
3  int main(){
4      int a = 2;
5      int* b = &a;
6      int* c;
7      c = b;
8      *c = 23;
9
10
11
12
13
14
15
16      return 0;
17 }
```

Lets inspect in more detail

```
1  #include <iostream>
2  using namespace std;
3  int main(){
4      int a = 2;
5      int* b = &a;
6      int* c;
7      c = b;
8      *c = 23;
9      cout << "Value of a = " << a << endl;
10     cout << "Value of b = " << b << endl;
11     cout << "Value of c = " << c << endl;
12
13     cout << "Adress of a = &a = " << &a << endl;
14     cout << "Adress of b = &b = " << &b << endl;
15     cout << "Adress of c = &c = " << &c << endl;
16     return 0;
17 }
```

```
> ./simplepointer
Value of a = 23
Value of b = 0x16fa46f48
Value of c = 0x16fa46f48
Adress of a = &a = 0x16fa46f48
Adress of b = &b = 0x16fa46f40
Adress of c = &c = 0x16fa46f38
```

Lets inspect in more detail

```
1  #include <iostream>
2  using namespace std;
3  int main(){
4      int a = 2;
5      int* b = &a;
6      int* c;
7      c = b;
8      *c = 23;
9      cout << "Value of a = " << a << endl;
10     cout << "Value of b = " << b << endl;
11     cout << "Value of c = " << c << endl;
12
13     cout << "Adress of a = &a = " << &a << endl;
14     cout << "Adress of b = &b = " << &b << endl;
15     cout << "Adress of c = &c = " << &c << endl;
16     return 0;
17 }
```

address
VALUE
variable

@line 4

0x16fa46f48
2
a [int]

0x16fa46f40
0x16fa46f48
b [int*]

@line 5

0x16fa46f38
xxxxxxxxxxx
c [int*]

@line 6

0x16fa46f38
0x16fa46f48
c [int*]

@line 7

0x16fa46f48
23
a [int]

@line 8

```
> ./simplepointer
Value of a = 23
Value of b = 0x16fa46f48
Value of c = 0x16fa46f48
Adress of a = &a = 0x16fa46f48
Adress of b = &b = 0x16fa46f40
Adress of c = &c = 0x16fa46f38
```

Thought!!!

Pointers as natural choice for array?

Unit-02.4

Case study..

```
1  #include <iostream>
2  using namespace std;
3
4  int main(int argc, char const *argv[])
5  {
6      int arr[] = {1, 2, 3, 4, 5, 6};
7      cout << "arr = " << arr << endl;
8      return 0;
9  }
```

```
> ./simplearray
arr = 0x16bdf2f30
```

If you were thinking this would print an entire array, it's time to review some basics....

Here, 'arr' seems some kind of address. Well if you are guessing if this is the address of the beginning of the array, then you are right.

Case study..

```
1  #include <iostream>
2  using namespace std;
3
4  int main(int argc, char const *argv[])
5  {
6      int arr[] = {1, 2, 3, 4, 5, 6};
7      cout << "address of first element of array = &arr[0] = " << &arr[0] << endl;
8      cout << "Value of arr, arr = " << arr << endl;
9      return 0;
10 }
```

```
address of first element of array = &arr[0] = 0x16fcba30
Value of arr, arr = 0x16fcba30
```

If you were thinking this would print an entire array, it's time to review some basics....

Here, 'arr' seems some kind of address. Well if you are guessing if this is the address of the beginning of the array, then you are right.

likewise..

```
1  #include <iostream>
2  using namespace std;
3
4  int main(int argc, char const *argv[])
5  {
6      int arr[] = {1, 22, 31,};
7      cout << "address of first element of arr = " << arr << endl;
8      cout << "address of first element of arr = " << arr + 1 << endl;
9      cout << "address of first element of arr = " << arr + 2 << endl;
10
11     /* verify this using address resolution,
12     i.e., use *address, so *arr should give us first element
13     similarly, *(arr+1) should give us second and so on
14     */
15     cout << "first element of arr = " << *arr << endl;
16     // In following you don't want to do *arr + 1,
17     // this will give you first element and add 1 to it
18     cout << "first element of arr = " << *(arr + 1) << endl;
19     cout << "first element of arr = " << *(arr + 2) << endl;
20     return 0;
21 }
```

```
> ./simplearray
address of first element of arr = 0x16d31af38
address of first element of arr = 0x16d31af3c
address of first element of arr = 0x16d31af40
first element of arr = 1
first element of arr = 22
first element of arr = 31
```

What is the key take away here?

```
int arr[]; // here arr holds address of first element of arr  
int* k; // here k holds address of some int
```

So you can treat array as a pointer.

so, this is possible. (traversing array using while loop instead of for loop)

```
1  #include <iostream>
2  using namespace std;
3
4  int main(int argc, char const *argv[])
5  {
6      int arr[] = {1, 22, 31,};
7      int size = sizeof(arr)/sizeof(arr[0]);
8      int* i = arr;
9
10     while(i < (arr + size)){
11         cout << *i << endl;
12         i++;
13     }
14     return 0;
15 }
```

```
> ./simplearray
1
22
31
```

Dynamic Memory Allocation

with examples

Unit-02.5

One-D array example

Illustrating Dynamic Memory Allocation

```
1  #include <iostream>
2  #include <random>
3  using namespace std;
4  int* oneDRandomArrayBetween(int, int, int);
5  void display(int*, int); // This is how you write a function that takes input
6
7  int* oneDRandomArrayBetween(int size, int low, int high){
8      /* This function generates random array of size, with low and high bounds*/
9      random_device rd;
10     mt19937 gen(rd()); // Standard mersenne_twister_engine seeded with rd()
11     // Define the distribution for integers between low and high inclusive
12     uniform_int_distribution<int> distribution(low, high);
13
14     // Define pointer now
15     int *arr = new int[size];
16     // Generate and print 10 random numbers
17     for (int i = 0; i < size; ++i) {
18         arr[i] = distribution(gen);
19     }
20     return arr;
21 }
```

Dynamic Memory Allocation

Dynamic memory allocation in C++ refers to the process of allocating memory at runtime from the heap (rather than the stack) for storing data. It allows for the creation and manipulation of data structures whose size is not known at compile time. This is typically achieved using the `new` keyword to allocate memory and the `delete` keyword to deallocate it when it's no longer needed. Dynamic memory allocation provides flexibility but also requires careful management to avoid memory leaks or undefined behavior.

```
23 void display(int* arr, int size){
24     cout << "arr = {";
25     for(int i = 0; i < size; i++) {cout << arr[i] << ", ";}
26     cout << "\b\b" << endl; // FUN : Do you know why \b\b here?
27 }
28
29 int main(){
30     int size = 10, low = 2, high = 4;
31     int* arr = oneDRandomArrayBetween(size, low, high);
32     display(arr, size);
33     return 0;
34 }
```

```
> ./simplepointer
arr = {4, 2, 3, 2, 4, 2, 2, 3, 4, 3}
```

Two-D array example

Illustrating Dynamic Memory Allocation | Generate 2x3 random array with low, high

```
1  #include <iostream>
2  #include <random>
3  using namespace std;
4  int** twoDRandomArrayBetween(int, int, int, int);
5  void display(int**, int, int); // This is how you write a function that takes input
6
7  int** twoDRandomArrayBetween(int row, int col, int low, int high){
8      // Random number setup
9      random_device rd;
10     mt19937 gen(rd());
11     uniform_int_distribution<int> distribution(low, high);
12
13     // IMPORTANT!!! You can only define how many rows you can have first
14     int** arr = new int*[row]; // Dynamic Memory Allocation
15     // Iterate over each row and define how many columns we need
16     for (int i = 0; i < row; ++i) {
17         // For each row you need to have fix size columns, here 'col'
18         // Following means for ith row we need col amount of data
19         arr[i] = new int[col]; // Dynamic Memory Allocation
20         for(int j = 0; j < col; ++j){arr[i][j] = distribution(gen);}
21     }
22     return arr;
23 }
```

Two Steps of Dyn.Mem.All

- First allocate number of rows
- Second allocate number of columns

What do you think we should do for 3D array?

Example – IMAGE

IMAGE in CG is represented as C x H x W, where,
C is number of color channels, RGB, which is 3
H is number of pixels in rows, or width of image
W is number of pixels in cols, or height of image

```
25 void display(int** arr, int row, int col){
26     cout << "arr = \n\t{" << endl;
27     for(int i = 0; i < row; i++) {
28         for(int j = 0; j < col; j++) {cout << "\t" << arr[i][j] << ", ";}
29         cout << "\b\b" << endl; // FUN : Do you know why \b\b here?
30     }
31     cout << "\t}" << endl;
32 }
33
34 int main(){
35     int row = 2, col = 3, low = 10, high = 99;
36     int** arr = twoDRandomArrayBetween(row, col, low, high);
37     display(arr, row, col);
38     return 0;
39 }
40
```

Test Your knowledge.

Find out why following is a stupid idea..

```
1  #include <iostream>
2  #include <random>
3  using namespace std;
4
5  int display(int arr[]){
6      size_t size = sizeof(arr)/sizeof(arr[0]);
7      for(int i=0; i < size; i++) { cout << arr[i] << endl; }
8  }
9
10 int main(){
11     int arr[] = {0,1,2,3,4,5,6};
12     display(arr);
13     return 0;
14 }
```

FYI

1. Stack vs Heap:

- The stack is a small, fast memory region for fixed-size, short-lived variables (e.g., local variables).
- The heap is a large, slower memory region for dynamically allocated data with flexible lifetimes.

2. Variable Length Arrays (VLAs):

- VLAs are not part of the C++ standard and are limited by the stack size. Using large VLAs can lead to stack overflows or undefined behavior.

3. Dynamic Memory Allocation in C++:

- The `new` keyword allocates memory on the heap, which must be explicitly freed with `delete` to prevent memory leaks.

4. Heap Advantages:

- The heap allows flexible memory allocation at runtime and is much larger than the stack, making it suitable for large arrays or objects.

5. Heap Risks in C++:

- Manual memory management can lead to bugs like memory leaks, dangling pointers, or double deletes if `delete` is not properly used.

6. Modern C++ Solutions:

- **Smart Pointers** (e.g., `std::unique_ptr`, `std::shared_ptr`) and containers (e.g., `std::vector`) automate heap memory cleanup, reducing the risk of leaks and bugs.

7. Garbage Collection in Other Languages:

- Languages like Python and Java use **garbage collection** to clean up unused heap memory automatically, relieving programmers from manual cleanup.

8. Python's Garbage Collection:

- Python uses **reference counting** and a **garbage collector** to handle memory cleanup, including resolving circular references.

9. C++'s Manual Cleanup Philosophy:

- C++ requires manual cleanup to give developers precise control over memory, optimizing performance and minimizing runtime overhead.

10. Best Practices:

- Avoid raw pointers in modern C++; prefer smart pointers or standard containers like `std::vector`.
- For critical systems requiring manual memory management, carefully track allocations and deallocations to avoid leaks and undefined behavior.